JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"Place to Learn, Chance to Grow"
2025-2026 (EVEN Semester)

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Branch: Artificial Intelligence **Semester: 6thSem 3rd Year**

Subject: LFTO

UNIT -III

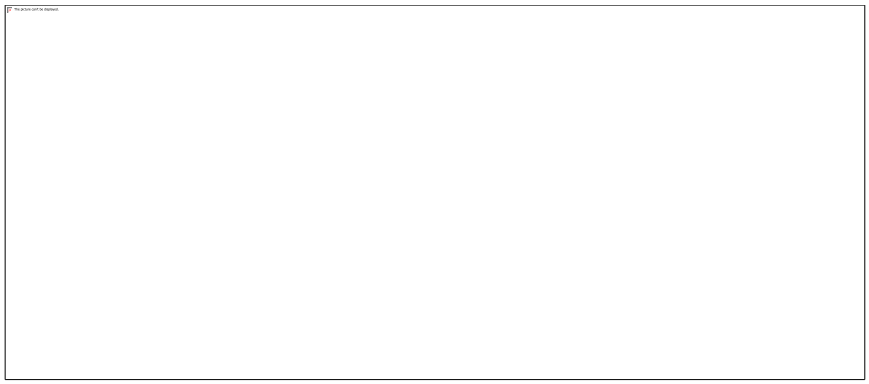
Parameter Efficient Fine-Tuning (PEFT): Concept and Need for PEFT, Methods: Adapters, Prefix Tuning, Prompt Tuning, Low-Rank Adaptation (LoRA) and its applications, Combining PEFT methods for efficient model customization

---

What is Parameter-Efficient Fine-Tuning (PEFT)?

Parameter-Efficient Fine-Tuning (PEFT) is a method used to fine-tune Large Language Models (LLMs) by updating a small subset of the model's parameter while keeping the majority of the pre-trained weights frozen. This makes fine-tuning much more efficient in terms of:

- **Computational cost**: Less computing power is required.
- **Storage**: Task-specific modules take up minimal storage.
- **Training time**: It's faster because fewer parameters are being updated.



Problem with Traditional Fine-Tuning

Fine-Tuning takes the pre-trained model and adapt the model for specific task. For example, BERT or GPT can be fine-tuned to perform sentiment analysis or text summarization. Traditionally, fine-tuning involves updating all the parameters (weights) of the model based on the new data. This works well but the problem is that modern Large Language Models (LLMs) are huge and have billions of parameters.

Imagine we have a **large language model** with 100 billion parameters. If we fine-tune all those parameters for every new task, we'll need a lot of computing power and storage. To solve this problem, researchers developed Parameter-Efficient Fine-Tuning (PEFT). With PEFT, we can achieve similar performance by tweaking only a small fraction of the model making it faster, cheaper and easier to manage while still giving strong performance.

Working of Parameter-Efficient Fine-Tuning

Let's see how it works step by step:

1. Start with a Pre-trained Model: We begin with a large language model (LLM) such as GPT, BERT or T5 that is already trained on massive amounts of text. This model contains general knowledge about language.

2. Freeze the Original Weights: Instead of updating all the billions of weights, we keep them fixed. This saves computing power and avoids the need to store multiple full versions of the model.

3. Add Lightweight Components: Since the main model is frozen, we introduce tiny, task-specific modules or allow updates only to certain parameters. These are the only parts that will be trained. Some common techniques are:

- **Adapters**: Small layers added between existing layers to capture task-specific adjustments.
- **LoRA (Low-Rank Adaptation):** Adds compact matrices that approximate updates without touching the main weights.
- **Prompt / Prefix Tuning**: Attaches short, learnable prompt vectors that guide the model's behavior for each task.

4. Train Only the New Parts: During fine-tuning, we update just the added or selected parameters and the frozen weights stays untouched.

5. Deploy Efficiently: For each new task, we only need to save the small trained modules not the full model which means:

- One big pre-trained model can serve many tasks.

- Switching tasks is as simple as loading the right lightweight module.

How does parameter-efficient fine-tuning work?

PEFT works by freezing most of the pretrained model parameters and layers while adding a few trainable parameters, known as adapters, to the final layers for predetermined downstream tasks.

The fine-tuned models retain all the learning gained during training while specializing in their respective downstream tasks. Many PEFT methods further enhance efficiency with gradient checkpointing, a memory-saving technique that helps models learn without storing as much information at once.

Why is parameter-efficient fine-tuning important?

Parameter-efficient fine-tuning balances efficiency and performance to help organizations maximize computational resources while minimizing storage costs. When tuned with PEFT methods, transformer-based models such as GPT-3, LLaMA and BERT can use all the knowledge contained in their pretraining parameters while performing better than they otherwise would without fine-tuning.

PEFT is often used during transfer learning, where models trained in one task are applied to a second related task. For example, a model trained in image classification might be put to work on object detection. If a base model is too large to completely retrain or if the new task is different from the original, PEFT can be an ideal solution.

PEFT versus fine-tuning

Traditional full fine-tuning methods involve slight adjustments to all the parameters in pretrained LLMs to adapt them for specific tasks. But as developments in artificial intelligence (AI) and deep learning have led models to grow larger and more complex, the fine-tuning process has become too demanding on computational resources and energy.

Also, each fine-tuned model is the same size as the original. All these models take up significant amounts of storage space, further driving up costs for the organizations that use them. While fine-tuning does create more efficient machine learning (ML), the process of fine-tuning LLMs has itself become inefficient.

PEFT adjusts the handful of parameters that are most relevant to the model's intended use case to deliver specialized model performance while reducing model weights for significant computational cost and time savings.

PEFT benefits

Parameter-efficient fine-tuning brings a wealth of benefits that have made it popular with organizations that use LLMs in their work:

- Increased efficiency
- Faster time-to-value
- No catastrophic forgetting
- Lower risk of overfitting
- Lower data demands
- More accessible AI
- More flexible AI

Increased efficiency
Most large language models used in generative AI (gen AI) are powered by expensive graphics processing units (GPUs) made by manufacturers such as Nvidia. Each LLM uses large amounts of computational resources and energy. Adjusting only the most relevant parameters imparts large savings on energy and cloud computing costs.

Faster time-to-value
Time-to-value is the amount of time that it takes to develop, train and deploy an LLM so it can begin generating value for the organization that uses it. Because PEFT tweaks only a few trainable parameters, it takes far less time to update a model for a new task. PEFT can deliver comparable performance to a full fine-tuning process at a fraction of the time and expense.

No catastrophic forgetting
Catastrophic forgetting happens when LLMs lose or "forget" the knowledge gained during the initial training process as they are retrained or tuned for new use cases. Because PEFT preserves most of the initial parameters, it also safeguards against catastrophic forgetting.

Lower risk of overfitting
Overfitting is when a model hews too closely to its training data during the training process, making it unable to generate accurate predictions in other contexts. Transformer models tuned with PEFT are much less prone to overfitting as most of their parameters remain static.

Lower data demands
By focusing on a few parameters, PEFT lowers the training data requirements for the fine-tuning process. Full fine-tuning requires a much larger training data set because all the model's parameters will be adjusted during the fine-tuning process.

More accessible AI
Without PEFT, the costs of developing a specialized LLM are too high for many smaller or medium-sized organizations to bear. PEFT makes LLMs available to teams who might not otherwise have the time or resources to train and fine-tune models.

More flexible AI
PEFT enables data scientists and other professionals to customize general LLMs to individual use cases. AI teams can experiment with model optimization without worrying as much about burning through computational, energy and storage resources.

PEFT Techniques for LLMs

PEFT is not a single method but a family of techniques. The choice of method depends on the task, resources and flexibility needed. Let's see some popular techniques used with large language models:

1. Adapter Modules

Adapters are one of the first PEFT techniques to be applied to natural language processing (NLP) models. Researchers strove to overcome the challenge of training a model for multiple downstream tasks while minimizing model weights. Adapter modules were the answer: small add-ons that insert a handful of trainable, task-specific parameters into each transformer layer of the model.

- Adapters Modules are small, trainable modules inserted between the layers of a pre-trained model. During fine-tuning, only the adapter modules are updated while the original model weights remain fixed. Once fine-tuned, it can be easily added or removed allowing for modular customization of the model.

- It allow for efficient multi-task learning where different adapters can be used for different tasks while sharing the same base model.

- For example: The Hugging Face AdapterHub provides an extensive library of pre-trained adapters for various NLP tasks.

Applications of Adapters in Parameter Efficient Fine-Tuning (PEFT)

Adapters are small trainable modules inserted into pre-trained models. They enable efficient task adaptation without modifying the entire model. Below are detailed real-world and academic applications.

1. Applications in Natural Language Processing (NLP)

1.1 Text Classification

Adapters are widely used for:

- Sentiment Analysis
- Spam Detection
- Fake News Detection
- Topic Classification
- Emotion Detection

How Adapters Help:

- Same base model (e.g., BERT) can support multiple classification tasks.
- Each task has its own adapter.
- Switching tasks = switching adapter (not entire model).

Example: A university may use:

- One adapter for student feedback sentiment analysis

- Another adapter for research paper classification

1.2 Named Entity Recognition (NER)

Adapters help fine-tune models for:

- Medical NER (diseases, drugs)
- Legal NER (case numbers, acts)
- Financial NER (currencies, stock names)

Advantage:

Domain-specific knowledge is captured without retraining the full model.

1.3 Question Answering Systems

Adapters are used to adapt LLMs for:

- Educational QA systems
- Technical support bots
- Customer service automation
- FAQ systems

Each domain can have a separate adapter.

1.4 Machine Translation

Adapters are used in multilingual transformer models to:

- Support low-resource languages
- Improve domain-specific translation
- Add new languages efficiently

Example:

- English → Hindi adapter
- English → Marathi adapter
- English → Technical domain translation adapter

2. Domain Adaptation Applications

Adapters are extremely powerful for domain adaptation.

2.1 Medical Domain

Used for:

- Clinical text analysis
- Radiology report summarization
- Medical chatbots

Medical adapter learns domain terminology while base model remains unchanged.

2.2 Legal Domain

Applications include:

- Contract analysis
- Legal document summarization
- Case law retrieval

Legal adapter captures legal-specific vocabulary and structure.

2.3 Financial Domain

Used for:

- Financial news classification
- Risk analysis
- Market sentiment prediction

Different financial institutions can use custom adapters.

3. Multilingual Applications

Adapters allow language-specific customization.

Example:

- One base multilingual model
- Hindi adapter
- Marathi adapter
- English adapter

Benefits:

- Efficient multilingual support
- No need for separate models per language
- Easy addition of new languages

4. Multi-Task Learning

Adapters support multiple tasks within a single model.

Example:

One base LLM +

- Adapter 1 → Summarization
- Adapter 2 → Translation
- Adapter 3 → Sentiment Analysis
- Adapter 4 → Question Answering

This reduces:

- Storage
- Deployment complexity
- Computational cost

5. Enterprise & Industry Applications

5.1 Customer Support Chatbots

Companies use:

- One base model
- Department-specific adapters

Example:

- HR adapter
- Finance adapter
- Technical support adapter

Switch adapter depending on query type.

5.2 Personalized AI Systems

Adapters can be:

- User-specific
- Region-specific
- Organization-specific

Example:
An EdTech platform can:

- Use a general LLM
- Add school-specific adapters

5.3 Edge and Cloud Deployment

Adapters are lightweight and can be:

- Easily stored
- Quickly loaded
- Switched dynamically

Ideal for:

- Mobile applications
- Cloud APIs
- SaaS platforms

6. Research and Academic Applications

Adapters are widely used in:

- Low-resource learning
- Continual learning
- Transfer learning
- Multi-domain evaluation
- Cross-lingual research

Researchers can:

- Share base model
- Share small adapter modules
- Reduce storage requirements

7. Continual Learning

Adapters help avoid catastrophic forgetting.

Instead of retraining model:

- Add new adapter for new task
- Old knowledge remains intact

Example:
Year 1 → Add classification adapter
Year 2 → Add summarization adapter
Year 3 → Add chatbot adapter

Base model remains stable.

8. Large Language Model (LLM) Customization

Adapters are used in:

- GPT-based systems
- LLaMA fine-tuning
- T5-based architectures

Applications:

- Instruction tuning
- Domain-specific chatbots
- Enterprise AI customization

9. Advantages of Using Adapters in Applications

✓ Very low training cost
✓ Easy task switching
✓ Scalable to multiple domains
✓ Reduces storage requirements
✓ Suitable for large models
✓ Supports modular AI systems

2. LoRA (Low-Rank Adaptation)

Introduced in 2021, low-rank adaption of large language models (LoRA) uses twin low-rank decomposition matrices to minimize model weights and reduce the subset of trainable parameters even further.

- LoRA (Low-Rank Adaptation) reduces the number of trainable parameters by decomposing weight updates into low-rank matrices. Instead of updating the entire weight matrix, it modifies only a small, low-rank component which approximates the changes needed for fine-tuning.

- It achieves results close to full fine-tuning but with far fewer parameters.

- It has been successfully applied to LLMs like GPT-3 and T5 making it a popular choice for parameter-efficient fine-tuning at scale.

LoRA (Low-Rank Adaptation) – Applications

Low-Rank Adaptation (LoRA) is one of the most widely used Parameter Efficient Fine-Tuning (PEFT) methods for customizing large pre-trained models efficiently. It enables fine-tuning by training only small low-rank matrices while keeping the base model frozen.

Below are detailed applications across various domains.

1. Applications in Large Language Models (LLMs)

LoRA is heavily used to fine-tune large models such as:

- GPT-based models
- LLaMA
- Mistral
- T5
- Falcon

Instead of updating billions of parameters, LoRA trains only small low-rank matrices.

1.1 Instruction Tuning

LoRA is used to adapt base LLMs to follow instructions.

Example:

- Fine-tuning a base LLM to answer in structured format
- Adapting model for academic writing
- Converting general LLM into domain-specific assistant

Why LoRA?

- Instruction tuning datasets are large
- Full fine-tuning is expensive
- LoRA reduces memory requirements

2. Text Summarization

LoRA is widely used for:

- News summarization
- Research paper summarization
- Legal document summarization
- Medical report summarization

Example:
A hospital may use:

- Base LLM
- Add medical LoRA module
- Generate concise clinical summaries

Benefits:

- Small training cost
- Domain-specific adaptation
- Easy deployment

3. Code Generation

LoRA is applied to adapt LLMs for programming tasks.

Applications:

- Python code generation
- Java/C++ support
- Debugging assistants
- Code completion systems

Example:
Fine-tuning a general LLM to specialize in:

- Web development
- Data science scripts
- Competitive programming

LoRA is popular in:

- AI coding assistants
- Software engineering tools

4. Chatbot Development

LoRA is widely used to build domain-specific chatbots.

4.1 Customer Support Bots

One base model can support:

- Banking chatbot
- Healthcare chatbot

- University admission chatbot

Each domain uses a separate LoRA module.

4.2 Conversational AI

LoRA helps:

- Control tone (formal/informal)
- Add personality
- Improve context understanding

5. Domain Adaptation

LoRA is highly effective in domain-specific model customization.

5.1 Medical AI

Applications:

- Clinical note analysis
- Disease prediction explanation
- Drug interaction QA

Benefits:

- Captures medical vocabulary
- Requires small domain dataset
- Efficient training

5.2 Legal AI

Applications:

- Contract summarization
- Case law retrieval
- Legal argument generation

Law firms can:

- Use one base model
- Add legal LoRA for customization

5.3 Financial AI

Applications:

- Financial report analysis
- Risk prediction explanation
- Stock sentiment analysis

Banks can maintain:

- Base LLM
- Finance-specific LoRA module

6. Multilingual Adaptation

LoRA can be used to:

- Add support for low-resource languages
- Improve translation quality
- Enhance cross-lingual performance

Example:

- English base model
- Add Hindi LoRA
- Add Marathi LoRA

This avoids retraining full model for each language.

7. Image Generation & Diffusion Models

LoRA is extremely popular in:

- Stable Diffusion customization
- Artistic style learning
- Character generation
- Personalized image models

Example:
Train LoRA on:

- Specific art style
- Specific face/person
- Specific product images

Result:

- Lightweight model file (few MB)
- Easy sharing
- No need to retrain full diffusion model

8. Speech and Multimodal Models

LoRA is applied in:

- Speech recognition fine-tuning
- Speech-to-text domain adaptation
- Vision-Language models
- Video understanding models

Example:

- Fine-tuning multimodal LLM for medical image explanation

9. Edge & Low-Resource Deployment

LoRA is suitable for:

- Limited GPU environments
- Academic research labs
- Startups with small budgets
- On-device AI systems

Because:

- Low memory footprint
- Reduced training cost
- Faster experimentation

10. Multi-Task Learning

LoRA supports:

One base model + multiple LoRA modules

Example:

- LoRA 1 → Summarization
- LoRA 2 → Code generation
- LoRA 3 → Chatbot
- LoRA 4 → Translation

Switch modules without retraining full model.

11. Continual Learning

LoRA helps in:

- Adding new tasks over time
- Avoiding catastrophic forgetting
- Maintaining base model stability

Instead of overwriting weights:

- Add new LoRA for new task

12. Enterprise AI Systems

Companies use LoRA for:

- Internal document search
- AI assistants for employees
- Knowledge base systems
- Automation tools

Advantages for enterprises:

- Low infrastructure cost
- Modular AI deployment
- Secure domain customization

13. Research Applications

LoRA is widely used in:

- Low-resource learning
- Transfer learning experiments
- Benchmark evaluations
- Model compression studies
- Efficient AI research

14. Advantages of LoRA in Applications

✓ Extremely parameter efficient
✓ Very low memory usage
✓ No need to modify base model
✓ Easy to store and share
✓ Scalable across tasks
✓ Minimal inference overhead
✓ Industry-friendly

3. Prefix Tuning

Specifically created for natural language generation (NLG) models, prefix-tuning appends a task-specific continuous vector, known as a prefix, to each transformer layer while keeping all parameters frozen. As a result, prefix-tuned models store over a thousandfold fewer parameters than fully fine-tuned models with comparable performance.

- Prefix tuning works by adding a small set of learnable "prefix" tokens to the model's input at every layer. These prefix tokens act as task-specific prompts that helps the model's behavior without changing its original parameters.

- It allows the model to retain its general knowledge while adapting to specific tasks through the learned prefixes.

- It is used for tasks like text generation where controlling the output style or content is important.

Applications of Prefix Tuning

3.1 Text Generation

Prefix Tuning works extremely well for generation tasks.

Applications include:

- Story generation
- Blog writing
- Content creation
- Dialogue generation
- Creative writing

Example:

A company can use:

- One prefix for marketing content
- Another prefix for technical writing
- Another prefix for creative storytelling

Only prefixes change — base model remains same.

3.2 Text Summarization

Prefix Tuning is widely used for:

- News summarization
- Research paper summarization
- Legal document summarization
- Medical report summarization

The prefix guides the model to focus on important content patterns during generation.

Benefit:

- Efficient domain adaptation without retraining full model.

3.3 Machine Translation

Prefix Tuning is applied to multilingual transformer models for:

- Language pair adaptation
- Low-resource language translation
- Domain-specific translation (technical/legal)

Example:

- English → Hindi prefix
- English → Marathi prefix
- Technical domain prefix

This reduces the need for full fine-tuning for each language pair.

3.4 Question Answering (QA)

Prefix Tuning helps in:

- Open-domain QA systems
- Educational Q&A platforms
- Customer support automation
- FAQ systems

Each application can have a task-specific prefix.

3.5 Conversational AI & Chatbots

Prefix Tuning is effective for building:

- Customer service bots
- Banking assistants

- E-commerce chatbots
- Academic guidance bots

Example:
One base LLM +

- HR prefix
- Finance prefix
- Technical support prefix

Switch prefix based on query type.

3.6 Domain Adaptation

Prefix Tuning is useful in domain-specific systems:

Medical Domain

- Clinical dialogue systems
- Health advisory bots
- Medical report interpretation

Legal Domain

- Contract analysis
- Legal document drafting
- Case summary generation

Financial Domain

- Market analysis reports
- Risk summary generation
- Investment advisory bots

Prefix vectors capture domain-specific patterns.

3.7 Low-Resource Learning

Prefix Tuning works well when:

- Training data is limited
- Full fine-tuning causes overfitting
- Computational resources are restricted

Because:

- Only small prefix parameters are trained
- Less risk of overfitting

3.8 Multi-Task Learning

Prefix Tuning supports multiple tasks using different prefixes.

Example:

Base Model +

- Prefix 1 → Summarization
- Prefix 2 → Translation
- Prefix 3 → Sentiment classification
- Prefix 4 → Dialogue generation

Benefits:

- No duplication of base model
- Efficient task switching

3.9 Personalization Systems

Prefix Tuning can be used for:

- User-specific response styles
- Organization-specific tone
- Region-specific customization

Example:
An EdTech platform may use:

- Prefix for school-level explanation
- Prefix for engineering-level explanation
- Prefix for competitive exam preparation

3.10 Research and Academic Applications

Prefix Tuning is used in:

- Transfer learning research
- Few-shot learning experiments
- Efficient fine-tuning studies
- Large model adaptation research

Researchers can test new tasks without modifying full model.

4. Prompt Tuning

Prompt-tuning simplifies prefix-tuning and trains models by injecting tailored prompts into the input or training data. Hard prompts are manually created, while soft prompts are AI-generated strings of numbers that draw knowledge from the base model. Soft prompts have been found to outperform human-generated hard prompts during tuning.

- Prompt tuning involves adding a set of learnable soft prompts to the input sequence. However, instead of modifying internal model layers, it operates completely at the input level making it even simpler to implement.
- It is lightweight and works well for tasks that require minimal changes to the model architecture.
- It works well in few-shot learning where we only have a small amount of labeled data and for tasks where quick adaptation is needed.

Prompt Tuning – Applications

Prompt Tuning is a Parameter Efficient Fine-Tuning (PEFT) method where soft prompts (trainable embeddings) are added to the input of a pre-trained model while keeping the model weights frozen. Instead of modifying internal weights, the model is guided using optimized prompts.
Below are the major applications of Prompt Tuning.
1. Applications in Large Language Models (LLMs)
Prompt Tuning is widely used with large models such as:

- OpenAI GPT models
- Google T5
- Meta LLaMA

Since these models are very large, updating all parameters is expensive. Prompt tuning allows adaptation by learning only small prompt vectors.

2. Text Classification
Prompt tuning can convert classification tasks into language modeling tasks.
Example:
Instead of training a classifier:
Input:
"The movie was amazing."

Prompt format:
"The sentiment of this movie is [MASK]."
The model predicts:
"positive"
Applications:

- Sentiment analysis
- Spam detection
- Topic classification
- Emotion detection

Benefit:
No need to modify internal model weights.

3. Question Answering (QA)
Prompt tuning is effective for:

- Educational QA systems
- FAQ chatbots
- Knowledge-based assistants
- Exam preparation tools

Example:
Prompt:
"Answer the following question in simple terms:"
It guides the model's response style.

4. Text Generation
Prompt tuning is heavily used in:

- Story generation
- Article writing
- Blog drafting
- Creative writing

Different prompts can control:

- Tone (formal/informal)
- Length (short/detailed)
- Style (technical/simple)

Applications:

- Content creation platforms
- Marketing copy generation
- Academic writing assistants

5. Dialogue Systems and Chatbots
Prompt tuning is widely used in conversational AI systems.

Applications:

- Customer support bots
- HR information systems
- Banking assistants
- E-commerce chatbots

Example:

A company can train different soft prompts for:

- Friendly tone
- Professional tone
- Technical tone

Switching prompts changes response style without retraining the model.

6. Domain Adaptation

Prompt tuning allows domain-specific customization.

6.1 Medical Domain

Prompt:

"Provide a medically accurate explanation:"

Used for:

- Symptom explanation
- Health Q&A systems

6.2 Legal Domain

Prompt:

"Summarize the legal implications of:"

Used for:

- Contract analysis
- Case law summaries

6.3 Financial Domain

Prompt:

"Explain the financial risk in simple terms:"

Used for:

- Financial advisory systems
- Market analysis summaries

7. Few-Shot and Zero-Shot Learning

Prompt tuning is extremely powerful in:

- Few-shot learning (small dataset)
- Zero-shot learning (no task-specific training)

Large models can generalize tasks based on well-designed prompts.

Applications:

- Rapid prototyping

- Low-resource language tasks
- Startup AI products

8. Multilingual Applications

Prompt tuning can adapt multilingual models for:

- Language-specific responses
- Translation tasks
- Regional chatbot systems

Example:

Prompt for translation:

"Translate the following sentence into Hindi:"

This avoids training separate translation models.

9. Personalization Applications

Prompt tuning supports:

- User-specific response styles
- Age-based explanation (kids vs adults)
- Academic level adaptation

Example:

Prompt:

"Explain this concept to a 10-year-old student."

Used in:

- EdTech platforms
- Personalized tutoring systems

10. Enterprise AI Systems

Organizations use prompt tuning for:

- Internal knowledge assistants
- Policy explanation bots
- Employee support systems
- Business analytics assistants

Benefits:

✓ Very low computational cost

✓ Easy deployment

✓ Fast task switching

✓ Lightweight storage

11. Research Applications

Prompt tuning is used in:

- Low-resource NLP research

- Transfer learning experiments
- Cross-domain evaluation
- Language understanding benchmarks

Researchers can:

- Share small prompt embeddings
- Reuse large pre-trained models

Comparison with Other PEFT Methods (Application Perspective)

Feature	Prompt Tuning	Adapters	LoRA
Modifies internal layers	No	Yes	Yes
Trainable parameters	Extremely Low	Low	Very Low
Best for	Large LLMs	Multi-task systems	Large-scale fine-tuning
Implementation complexity	Simple	Moderate	Moderate